

INSTALLATION AND CONFIGURATION GUIDE

Project: A Microservices-Based Architectural Approach to System Observability and Resilience

Scope: User, Order, and Payment Services Deployment with ELK Stack & Zipkin

1. Introduction

This document provides a comprehensive guide for deploying the microservices ecosystem. The deployment strategy utilizes **Containerization** to ensure environmental parity across different operating systems. By following this guide, the entire infrastructure—comprising three microservices and four observability components—can be instantiated as a unified cluster.

2. System Requirements

Before initiation, ensure the host machine meets the following minimum specifications to handle the resource-intensive ELK Stack:

- **Operating System:** Windows 10/11 (with WSL2), macOS, or Linux (Ubuntu 20.04+ recommended).
- **Processor:** Minimum 4 Cores (Intel i5/Ryzen 5 or higher).
- **Memory (RAM):** 16 GB (8 GB is the absolute minimum, but 16 GB is required for smooth ELK indexing).
- **Disk Space:** 10 GB of free space.
- **Software Dependencies:**
 - **Docker Desktop** (version 4.x+) or **Docker Engine**.
 - **Docker Compose** (version 2.x+).
 - **Java JDK 17** (only required for local development/compilation).

3. Infrastructure Components

The orchestration file (`docker-compose.yml`) initializes the following containers:

1. **Application Tier:** `user-service`, `order-service`, `payment-service`.
2. **Tracing Tier:** `zipkin`.
3. **Logging Tier:** `elasticsearch`, `logstash`, `kibana`.

4. Step-by-Step Deployment

Phase 1: Environment Preparation

1. Clone the repository to your local machine:

Bash

```
Git clone https://github.com/busra106/microservices-distributed-  
tracing-system.git  
cd microservices-distributed-tracing-system
```

2. Verify Docker status:

Bash

```
docker info
```

Phase 2: Building Microservices

Each service must be packaged into a JAR file before being containerized (assuming Maven is used):

```
# Execute for each service directory  
mvn clean package -DskipTests
```

Note: If the Dockerfile is configured for multi-stage builds, this step is handled automatically during the image creation phase.

Phase 3: Orchestration with Docker Compose

Execute the following command in the root directory where the `docker-compose.yml` file is located:

```
docker-compose up -d --build
```

- `-d`: Runs containers in detached mode.
- `--build`: Ensures that the latest code changes are reflected in the images.

5. Service Access & Port Configuration

Once the containers are operational, the following endpoints will be accessible:

Component	Port	Endpoint
User Service	8081	http://localhost:8081
Order Service	8082	http://localhost:8082
Payment Service	8083	http://localhost:8083

Component	Port	Endpoint
Zipkin UI	9411	<code>http://localhost:9411</code>
Kibana UI	5601	<code>http://localhost:5601</code>
Elasticsearch	9200	<code>http://localhost:9200</code>

6. Post-Installation Validation

6.1. Health Check

Check the status of all containers:

```
docker-compose ps
```

All states should be `Up` or `Healthy`.

6.2. Verifying Observability Pipelines

1. **Zipkin Validation:** Access `http://localhost:9411` and perform a "Run Query" to ensure the services are registering their traces.
2. **Kibana Configuration:**
 - Navigate to `Stack Management > Index Patterns`.
 - Create a new index pattern (e.g., `microservices-*`) to visualize logs flowing from Logstash.

7. Troubleshooting

- **Elasticsearch Exit Code 137:** This indicates an Out of Memory (OOM) error. Increase the Docker Desktop memory limit in `Settings > Resources`.
- **Port Conflict:** If ports 8081-8083 are in use, modify the `ports` mapping in the `docker-compose.yml` file.
- **Logstash Connection Refused:** Ensure Elasticsearch is fully "Yellow" or "Green" status before Logstash starts. Docker Compose `depends_on` with `condition: service_healthy` is recommended.

8. Conclusion

The installation is complete when all microservices can communicate and logs are searchable within Kibana. For further technical details regarding the fault injection scenario, refer to the **Project Paper (Section 5)**.